NOTE Remember, all the features outlined in this chapter are specific to Razor, which is a server-side rendering engine. If you're using a client-side single-page application (SPA) framework to build your application, you'll likely handle these requirements in other ways, within the client.

Sections provide a way of organizing where view elements should be placed within a layout. They're defined in the view using an `@section` definition, as shown in the following listing, which defines the HTML content for a sidebar separate from the main content, in a section called `Sidebar`. The `@section` can be placed anywhere in the file, top or bottom, wherever is convenient.

Listing 17.10 Defining a section in a view template

```
@{
    Layout = "_TwoColumn";
}
@section Sidebar {                          ❶
    <p>This is the sidebar content</p>      ❶
}                                           ❶
<p>This is the main content </p>            ❷
```

❶ All content inside the braces is part of the Sidebar section, not the main body content.

❷ Any content not inside an @section will be rendered by the @RenderBody call.

The section is rendered in the parent layout with a call to `@RenderSection()`. This renders the content contained in the child section into the layout. Sections can be either required or optional. If they're required, a view must declare the given `@section`; if they're optional, they can be omitted, and the layout will skip them. Skipped sections won't appear in the rendered HTML. The following listing shows a layout that has a required section called `Sidebar` and an optional section called `Scripts`.

Listing 17.11 Rendering a section in a layout file, _TwoColumn.cshtml